MANUAL INTEGRAL

Soccer Edge System

Estimación de probabilidad, edge y seguimiento de pronósticos

Guía de usuario · Instalación · Arquitectura · Mantenimiento

Versión del documento: 1.0 · Agosto 2026
Proyecto documentado: Pipeline analítico cg0p
Autor y responsable del proyecto: Jimmy Toapanta
Fecha de revisión: 13/08/2026

Control del documento

Campo	Detalle
Proyecto	Soccer Edge System
Alcance	Manual de usuario y referencia técnica en un único documento
Versión documentada	Pipeline analítico cg0p
Rama de referencia	main
Estado	Sistema de análisis; modelo, datos y resultados se mantienen fuera de Git
Público previsto	Analistas deportivos y mantenedores de modelos Python
Responsable	Jimmy Toapanta
Última revisión	13/08/2026

Cómo usar este manual

La primera mitad acompaña a quien instala y utiliza la solución. La segunda explica cómo está construida, dónde guarda información y qué revisar antes de modificarla o desplegarla.

Este documento distingue lo que funciona hoy de las mejoras previstas. Cuando una integración depende de terceros, la guía lo indica y propone una verificación antes de operar con datos reales.

Contenido

Control del documento 2

Cómo usar este manual . . . 2

Contenido 3

El proyecto en pocas palabras 6

Propósito y alcance . . . 6

Qué resuelve . . . 6

Estado actual . . . 6

Alcance de esta entrega . . . 6

Fuera de alcance . . . 6

Preparación y puesta en marcha 7

Requisitos previos . . . 7

Instalación o incorporación . . . 7

Primera ejecución . . . 7

Operación paso a paso 8

Generar análisis de jornada . . . 8

Interpretar PICK y PASS . . . 8

Resolver pronósticos . . . 8

Evaluar una versión del modelo . . . 9

Lista de control operativa . . . 9

Problemas frecuentes y recuperación 10

Uso responsable, privacidad y seguridad . . . 10

Preguntas habituales . . . 10

¿Un edge positivo garantiza ganar? . . . 10

¿Dónde pongo la API key? . . . 10

¿Por qué se excluyen modelos y datos? . . . 10

Arquitectura y componentes **11**

Vista de arquitectura . . . 11

Responsabilidad de los componentes . . . 11

Configuración, datos e integraciones **12**

Parámetros importantes . . . 12

Persistencia y archivos generados . . . 12

Integraciones externas . . . 12

Flujos internos y decisiones de diseño **13**

Decisión . . . 13

Evaluación posterior . . . 13

Criterios de diseño . . . 13

Desarrollo, pruebas y mantenimiento **14**

Entorno de desarrollo . . . 14

Estrategia de pruebas . . . 14

Registro, diagnóstico y soporte . . . 14

Mantenimiento recomendado . . . 14

Despliegue y continuidad **15**

Ruta de despliegue . . . 15

Respaldo y recuperación . . . 15

Criterios de aceptación . . . 15

Límites conocidos y evolución **16**

Límites de la versión documentada 16

Hoja de ruta sugerida 16

Versiones y ramas 16

Entrega a otra persona **17**

Paquete de traspaso 17

Glosario breve 17

Cierre 17

01 · CONTEXTO

El proyecto en pocas palabras

Soccer Edge System reúne extracción de datos, construcción de características, estimación probabilística, eliminación de margen, reglas de decisión, pruebas de estrés y seguimiento de pronósticos. Su salida es analítica: no promete aciertos, rentabilidad ni elimina el riesgo de apostar.

Propósito y alcance

Hacer explícitas las hipótesis detrás de una selección: probabilidad del modelo, cuota, margen, edge, fracción de Kelly y desempeño histórico. El proyecto debe evaluarse por calibración y reproducibilidad, no por una racha corta de resultados.

Qué resuelve

- Pipeline modular de probabilidades y decisión.
- Poisson, de-vig, índice de caos y pruebas de estrés.
- Regla de pick/pass y staking fraccional.
- Seguimiento con Brier score y métricas históricas.
- Separación de datos, modelos y reportes generados.

Estado actual

La rama main contiene el pipeline actual. SHARP_API_KEY se lee desde el entorno en cg0p/config/settings.py; esa es la fuente autorizada aunque un texto antiguo sugiera editar config.py. El modelo entrenado y los datos no se incluyen, por lo que una instalación limpia necesita artefactos compatibles o un proceso de entrenamiento.

Alcance de esta entrega

- Instalación o incorporación de la versión presente en la rama principal.
- Recorridos de uso, verificaciones y recuperación ante fallos habituales.
- Arquitectura, configuración, datos, integraciones, pruebas y despliegue.
- Límites conocidos y una ruta de evolución coherente con el estado del código.

Fuera de alcance

- Garantizar ganancias, recuperar pérdidas o recomendar endeudamiento.
- Presentar una probabilidad como certeza o usar datos sin licencia.
- Confirmar modelos, bases o reportes generados dentro del repositorio.

02 · GUÍA DE USUARIO

Preparación y puesta en marcha

Utilice primero modo de análisis y registro. No arriesgue dinero para comprobar que la aplicación funciona: valide datos, cuotas, calibración y resolución histórica con una muestra independiente.

Requisitos previos

- Python y dependencias de requirements.txt.
- SHARP_API_KEY como variable de entorno cuando la fuente correspondiente esté habilitada.
- Datos/modelo compatibles, obtenidos legalmente y fuera de Git.
- Criterio de bankroll y límites personales independientes del programa.

Instalación o incorporación

1. Crear un entorno virtual e instalar requisitos.
2. Definir SHARP_API_KEY en la sesión, no en config.py.
3. Ubicar datos y modelo en las rutas locales ignoradas.
4. Ejecutar una corrida sin apostar.
5. Resolver pronósticos anteriores y revisar métricas antes de confiar en la salida.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python -m pip install -r requirements.txt
$env:SHARP_API_KEY='VALOR_LOCAL_NO_VERSIONADO'
python main.py
python main.py resolver
```

Primera ejecución

1. Confirmar que la fuente de datos y la fecha son correctas.
2. Ejecutar python main.py.
3. Revisar probabilidades, cuotas y motivos de PASS.
4. No usar un pick si falta información o el modelo no corresponde a la versión.
5. Ejecutar resolver sobre casos históricos y revisar Brier/calibración.

Señal de que todo quedó bien: el pipeline produce decisiones explicables, permite reconstruir los datos usados y actualiza métricas sin considerar un resultado aislado como validación.

Operación paso a paso

Los recorridos siguientes están escritos para una jornada real de trabajo. Antes de confirmar una acción masiva, conviene validar primero un caso de prueba y revisar el resultado.

Generar análisis de jornada

1. Actualizar datos autorizados.
2. Verificar equipo, competición, hora y cuota.
3. Ejecutar main.py.
4. Revisar probabilidad real estimada, edge y prueba de estrés.
5. Separar PICKS de PASS y conservar las razones.

Resultado esperado: cada decisión muestra insumos y umbrales, no solo una etiqueta.

Interpretar PICK y PASS

1. Comprobar que la cuota está entre 1.05 y 4.0.
2. Comparar edge con el mínimo 3.5.
3. Confirmar probabilidad real de al menos 20%.
4. Revisar stake de Kelly fraccional 0.25 y tope 4%.
5. Tratar PASS como una decisión válida.

Resultado esperado: no se fuerza una jugada cuando los criterios no se cumplen.

Resolver pronósticos

1. Confirmar resultados por una fuente confiable.
2. Ejecutar python main.py resolver.
3. Revisar correspondencia de evento/mercado.
4. Actualizar historial y métricas.
5. Analizar Brier y calibración en un periodo suficiente.

Resultado esperado: el historial conserva predicción y resultado sin editar el pronóstico después del partido.

Evaluar una versión del modelo

1. Congelar datos, código y parámetros.
2. Separar entrenamiento, validación y prueba temporal.
3. Comparar contra una referencia simple y cuotas sin margen.
4. Medir calibración, Brier y estabilidad.
5. Promover solo si mejora fuera de muestra.

Resultado esperado: la versión se acepta por evidencia reproducible, no por selección retrospectiva de aciertos.

Lista de control operativa

- Trabajar primero con datos de prueba cuando se cambie la configuración.
- Confirmar el resultado antes de ejecutar acciones masivas o irreversibles.
- Registrar fecha, entrada, salida y mensaje exacto cuando ocurra un error.
- Cerrar sesiones y proteger credenciales al terminar la jornada.

Problemas frecuentes y recuperación

Si algo falla, conserve el mensaje exacto, la hora y la acción que lo produjo. Esa información suele ser más útil que reinstalar o borrar la configuración sin diagnóstico.

Síntoma	Causa probable	Qué revisar
Falta SHARP_API_KEY	Variable de entorno ausente	Definirla en la sesión; no editar ni confirmar secretos.
No aparece el modelo	Artefacto local excluido	Obtener/entrenar versión compatible y registrar su huella.
Datos de partido no coinciden	Fuente, zona horaria o nombres	Validar identificadores y fecha antes de calcular.
Todo queda PASS	Umbrales, cuotas o probabilidades insuficientes	No bajar límites solo para producir picks; revisar insumos.
Métricas cambian demasiado	Muestra pequeña o fuga temporal	Ampliar ventana y auditar partición de datos.

Uso responsable, privacidad y seguridad

- Nunca confirmar contraseñas, tokens, claves API, perfiles del navegador ni archivos de configuración locales.
- Usar cuentas y permisos con el alcance mínimo necesario; retirar accesos cuando ya no se utilicen.
- Anonimizar correos, nombres, identificadores, órdenes y capturas antes de publicar evidencias en el portafolio.
- Revisar el historial completo de Git, no solo la última versión, antes de hacer público el repositorio.
- Cumplir legislación local, edad mínima y términos de las fuentes; detenerse si apostar causa daño.
- No presentar la herramienta como garantía financiera ni perseguir pérdidas aumentando el bankroll.

Preguntas habituales

¿Un edge positivo garantiza ganar?

No. Es una diferencia estimada entre probabilidad y cuota; el resultado individual sigue siendo incierto y el modelo puede estar mal calibrado.

¿Dónde pongo la API key?

En SHARP_API_KEY del entorno. La indicación antigua de editar config.py queda superada por settings.py.

¿Por qué se excluyen modelos y datos?

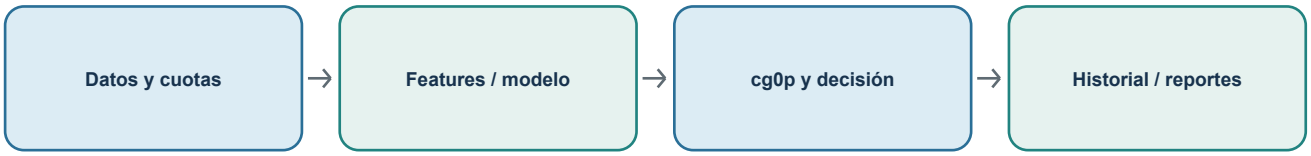
Por tamaño, licencias, privacidad y reproducibilidad; deben identificarse por versión/huella fuera de Git.

03 · REFERENCIA TÉCNICA

Arquitectura y componentes

El pipeline combina módulos de datos y características con un núcleo cg0p de probabilidad, de-vig, caos, estrés y decisión. Tracking separa la predicción emitida de su resolución posterior.

Vista de arquitectura



La decisión PICK/PASS se produce después de validar cuota, edge, probabilidad y estrés. La resolución ocurre más tarde y nunca debe reescribir la predicción original.

Responsabilidad de los componentes

Componente	Responsabilidad	Tecnología o archivo
Ingesta	Obtención y normalización	scraper, database
Modelo	Características y predicción	features, model
Núcleo cg0p	Poisson, de-vig, caos, estrés y decisión	cg0p/core/*
Módulos	Etapas A/B/C/D/F/G	modules/*
Tracking	Brier, historial y métricas	tracking/*
Presentación	CLI, GUI y reportes	main.py, gui, reportes

Configuración, datos e integraciones

Parámetros importantes

Parámetro / recurso	Uso	Tratamiento recomendado
SHARP_API_KEY	Acceso a fuente de datos	Variable de entorno; nunca config.py.
EDGE_MINIMO=3.5	Umbral de valor	Cambiar solo con evaluación fuera de muestra.
KELLY_FRACCION=0.25	Reducción de Kelly	No elimina riesgo.
BANKROLL=1000	Base de simulación	No equivale a dinero disponible.
VALUE_BET_MAXIMO=4.0	Tope porcentual	Límite de exposición, no garantía.
Modelo/datos	Artefactos analíticos	Versionar huellas y procedencia, no archivos sensibles.

Persistencia y archivos generados

- Datos, modelos, base, resultados y reportes se mantienen ignorados.
- Cada corrida debería registrar fecha, fuente, versión de modelo, parámetros y huellas.
- El historial de picks es inmutable: correcciones se anotan, no se sustituyen.
- Revisar licencias y términos antes de conservar o redistribuir datos.

Integraciones externas

Integración	Función	Riesgo operativo
Fuente Sharp	Datos o cuotas	API key, cuota y esquema.
Fuentes deportivas	Resultados y variables	Licencia, cobertura y correcciones.
Modelo local	Probabilidad estimada	Deriva, fuga y versión incompatible.
Reportes	Salida analítica	Interpretación fuera de contexto.

Flujos internos y decisiones de diseño

Decisión

1. Validar entrada y cuota 1.05–4.0.
2. Estimar probabilidad y retirar margen.
3. Calcular edge y estrés.
4. Descartar si edge <3.5 o probabilidad <20%.
5. Calcular Kelly fraccional y aplicar tope 4%.
6. Emitir PICK o PASS con razones.

Los números son parámetros de esta versión, no recomendaciones universales.

Evaluación posterior

1. Congelar la predicción antes del evento.
2. Incorporar resultado verificado.
3. Calcular Brier y métricas agregadas.
4. Segmentar por competición/mercado sin selección retrospectiva.
5. Revisar calibración y deriva.

La rentabilidad sola puede ocultar mala calibración en una muestra corta.

Criterios de diseño

- Separar configuración y credenciales del código que se versiona.
- Mantener una entrada clara para operar y otra para diagnóstico o pruebas.
- Preferir fallos explícitos a continuar con datos parciales o inventados.
- Conservar trazabilidad suficiente para reconstruir qué ocurrió sin exponer información sensible.

Desarrollo, pruebas y mantenimiento

Entorno de desarrollo

- Windows, Python compatible con el proyecto y un entorno virtual dedicado.
- Dependencias instaladas desde el archivo fijado por el repositorio.
- Git para aislar cambios y conservar un historial verificable.

```
python -m compileall .
python main.py
python main.py resolver
```

Estrategia de pruebas

- Pruebas unitarias para de-vig, Poisson, Kelly y límites.
- Backtest temporal sin mezclar información futura.
- Comparación contra baselines y probabilidades de mercado.
- Pruebas de estrés ante datos faltantes, cuotas extremas y deriva.

Registro, diagnóstico y soporte

- Conservar el mensaje completo y la hora del evento; evitar capturas que muestren credenciales o datos personales.
- Distinguir un error local de un fallo del servicio externo comprobando conectividad, permisos y cuotas.
- Repetir el caso con una entrada pequeña y controlada antes de cambiar dependencias o borrar datos.

Mantenimiento recomendado

- Revisar dependencias y servicios externos de manera periódica, probando la actualización en una rama separada.
- Actualizar este manual cuando cambien comandos, configuración, estructura de datos o flujo de usuario.
- Mantener ejemplos sin secretos y una prueba mínima que permita validar la instalación.
- Etiquetar versiones estables y documentar las alternativas como ramas, no como reemplazos ambiguos.

Despliegue y continuidad

Ruta de despliegue

1. Clonar o copiar una versión etiquetada en un equipo de prueba.
2. Crear un entorno virtual nuevo e instalar dependencias desde el archivo del proyecto.
3. Aplicar configuración local y credenciales fuera de Git.
4. Ejecutar una prueba pequeña; revisar salida, datos y registros.
5. Habilitar el acceso al usuario final solo después de la validación.

Respaldo y recuperación

- Respaldar solo datos necesarios y cifrar cualquier copia que contenga información personal u operativa.
- Probar la restauración en una ubicación distinta antes de confiar en el respaldo.
- Conservar una copia de la configuración sin secretos y registrar por separado dónde se administran las credenciales.

Criterios de aceptación

- Una instalación limpia puede completar el recorrido principal descrito en este manual.
- Los errores de credenciales, permisos o red se informan sin filtrar secretos.
- Los archivos temporales y los datos generados permanecen fuera del repositorio.
- La persona responsable puede identificar versión, configuración y evidencia de la última ejecución correcta.

Límites conocidos y evolución

Límites de la versión documentada

- El repositorio no incluye el modelo ni los datos necesarios para reproducir todos los resultados.
- El deporte es incierto y los patrones históricos cambian.
- Fuentes y cuotas pueden contener errores o actualizarse.
- No existe garantía de ganancia ni sustituto de una decisión responsable.

Hoja de ruta sugerida

Prioridad	Mejora	Resultado esperado
Alta	Manifiesto de datos/modelo con huellas	Reproducir una corrida sin publicar artefactos.
Alta	Suite matemática y backtest temporal	Validar reglas y evitar fuga.
Media	Reporte de calibración y deriva	Detectar deterioro antes de usar resultados.
Media	Modo estrictamente educativo por defecto	Separar análisis de cualquier ejecución financiera.

Versiones y ramas

Rama / versión	Papel en el historial	Recomendación
main · cg0p	Pipeline actual	Usar para evaluación reproducible.
alternativa/apuestas-auto-v3	Versión histórica alternativa	Conservar como referencia; no comparar sin mismo conjunto temporal.

Entrega a otra persona

Este apartado resume lo mínimo que debe conocer quien reciba el mantenimiento. La intención es que el proyecto pueda retomarse sin depender de la memoria del autor.

Paquete de traspaso

- Repositorio, rama y etiqueta exacta que se consideran estables.
- Lista de servicios externos, responsables de las cuentas y procedimiento para rotar accesos.
- Configuración de ejemplo sin secretos y ubicación de los datos persistentes.
- Resultado de la última prueba, limitaciones abiertas y próximos cambios recomendados.
- Un caso de prueba anonimizado que permita recorrer el flujo de principio a fin.

Glosario breve

Término	Significado en este proyecto
Edge	Diferencia entre probabilidad estimada y la implícita en la cuota.
De-vig	Retiro aproximado del margen de la casa.
Kelly	Fórmula de tamaño de exposición; aquí se usa una fracción.
Brier score	Error cuadrático de probabilidades; menor suele ser mejor.
Calibración	Correspondencia entre probabilidad anunciada y frecuencia observada.

Cierre

> *La mejor señal de madurez no es que el proyecto nunca falle, sino que otra persona pueda instalarlo, entender sus límites y recuperar una operación sin improvisar.*

Documento mantenido junto al código fuente. Las decisiones operativas que cambien el comportamiento deben reflejarse tanto en el código como en esta guía.